Raspberry Pi verstehen: Schritt für Schritt

Stand: Oktober 2026 · Version 63

Dieses Lernheft zeigt dir, was ein Raspberry Pi ist, wie du ihn in Betrieb nimmst und wie du mit Python LEDs, Taster und Sensoren steuerst. Die Beispiele sind für den Pi gedacht (Programme ausprobieren musst du am echten Gerät). Die Verbindung zum Arduino und zur Simulation im Tonfunk Lab zeigt dir das letzte Kapitel.

Inhalt

1. Was ist ein Raspberry Pi?
2. Inbetriebnahme
3. Die GPIO-Leiste
4. Python in fünf Minuten
5. Erstes Projekt: LED und Taster
6. Mehr Bauteile
7. Projektideen für den Pi
8. Fehler finden und sicher arbeiten
9. Arduino oder Pi? Und wie geht es hier weiter

1. Was ist ein Raspberry Pi?

Ein **Raspberry Pi** ist ein vollständiger kleiner **Computer** in der Größe einer Kreditkarte. Er hat Prozessor, Arbeitsspeicher, USB, HDMI, WLAN und läuft mit einem **Betriebssystem** (Raspberry Pi OS, ein Linux). Das Programm liegt nicht fest im Chip, sondern auf einer **SD-Karte**, und du startest es wie auf einem PC.

	Arduino Uno	Raspberry Pi
Art	Mikrocontroller	Einplatinen-Computer
Betriebssystem	keins, ein Programm	Linux, viele Programme gleichzeitig
Sprache	C++ (Sketch)	meist Python
Pin-Spannung	**5 V**	**3,3 V**
Analoge Eingänge	ja (A0 bis A5)	**nein** (ADC-Baustein nötig)
Zeitgenauigkeit	sehr genau, in Echtzeit	nicht garantiert (Linux läuft nebenher)
Strombedarf	~50 mA	500 mA bis 1,5 A, eigenes Netzteil
Starten	sofort	braucht Sekunden bis Minuten
Stärke	Sensoren und Motoren direkt steuern	Netzwerk, Kamera, Bildschirm, Datenbanken, KI

> **Merke:** Arduino = Steuern in Echtzeit. Pi = Rechnen, Vernetzen, Anzeigen. Beide arbeiten oft zusammen: Der Arduino liest den Sensor, der Pi wertet aus und zeigt an.

Zur Abgrenzung: Der **Raspberry Pi Pico** ist kein Pi-Computer, sondern ein Mikrocontroller wie der Arduino (Programmierung mit MicroPython oder C++).

2. Inbetriebnahme

Was du brauchst

- Raspberry Pi (zum Beispiel Pi 4 oder Pi 5) mit passendem **Netzteil** (Pi 4: 5 V / 3 A USB-C, Pi 5: 5 V / 5 A)
- **microSD-Karte** (mindestens 16 GB)
- Entweder Monitor (Micro-HDMI), Tastatur und Maus, oder nur WLAN und einen zweiten Computer (*headless*)

Betriebssystem auf die SD-Karte

1. Auf einem Computer das kostenlose Programm **Raspberry Pi Imager** installieren (raspberrypi.com/software).
2. Gerät (Raspberry Pi 4/5 …) und **Raspberry Pi OS** (64-Bit) wählen, SD-Karte auswählen.
3. Vor dem Schreiben auf **Einstellungen bearbeiten** klicken: Hostname (zum Beispiel `pi-lab1`), Benutzername und Passwort, WLAN-Zugang, Zeitzone und **SSH aktivieren**.
4. Schreiben lassen, SD-Karte in den Pi stecken, Netzteil anschließen. Der erste Start dauert 1 bis 2 Minuten.

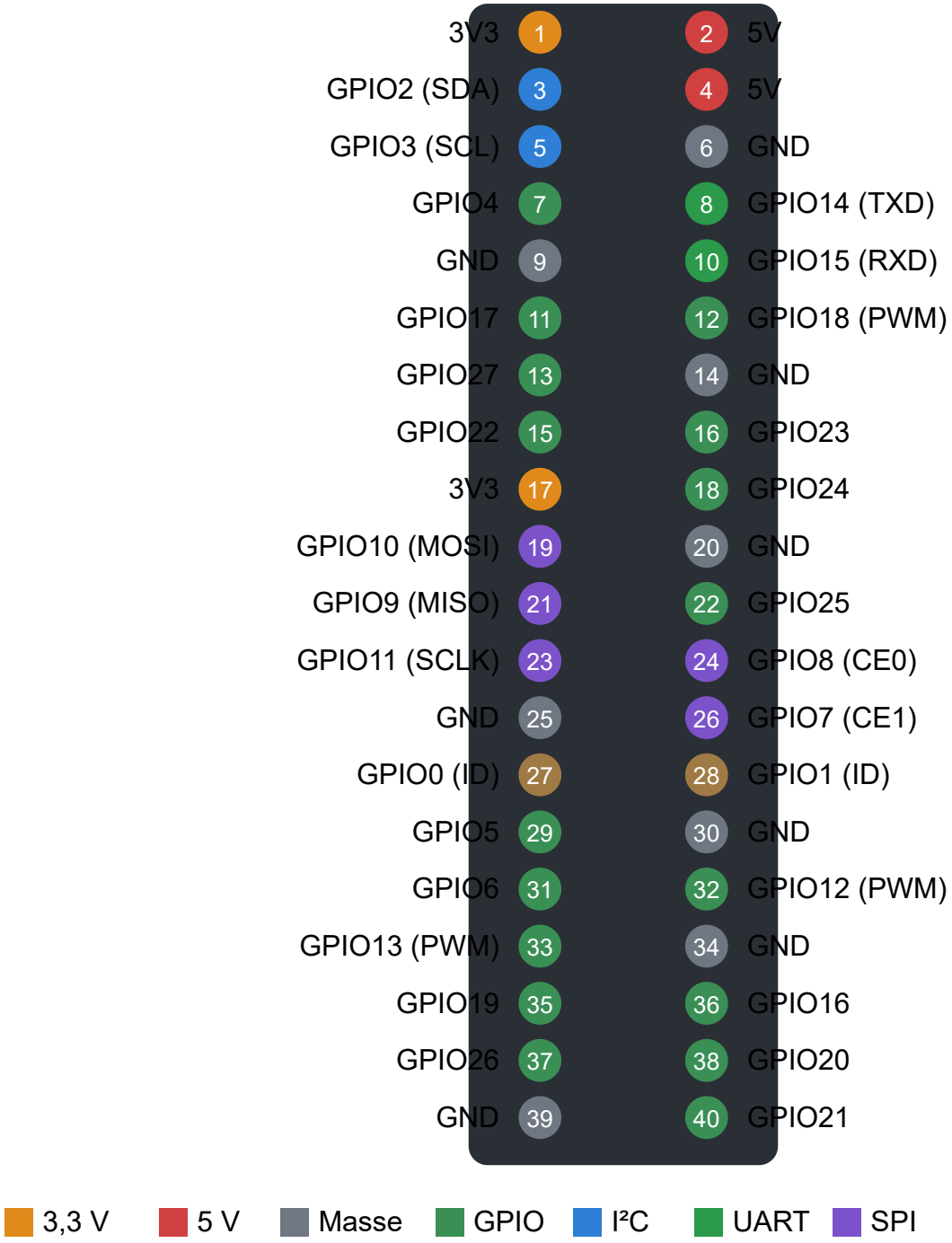
Verbinden

- **Mit Monitor:** Der Desktop startet. Du findest dort das Terminal und die Programmierumgebung **Thonny** (Menü Entwicklung).
- **Ohne Monitor (SSH):** Vom Computer oder **iPad** mit einer SSH-App (zum Beispiel Termius oder Blink) verbinden: `ssh benutzername@pi-lab1.local`.
- **Fernsteuerung des Desktops:** Mit „Raspberry Pi Connect“ oder einer VNC-App kannst du den Desktop auch am iPad anzeigen.

> **Wichtig:** Den Pi nie einfach vom Strom trennen, das kann die SD-Karte beschädigen. Erst herunterfahren: `sudo shutdown now` (oder im Desktop-Menü „Herunterfahren“), dann nach einigen Sekunden Netzteil abziehen.

3. Die GPIO-Leiste

Über die 40-polige Stiftleiste (**GPIO** = General Purpose Input/Output) verbindet der Pi sich mit der Elektronik. Die meisten Pins sind frei belegbare Ein-/Ausgänge, einige haben Sonderfunktionen (I^2C, UART, SPI, PWM).



Zwei Zählweisen

Jeder Pin hat eine **Pin-Nummer** (1 bis 40, wie auf der Leiste) und eine **GPIO-Nummer** (BCM). Das Python-Modul `gpiozero` arbeitet mit den **GPIO-Nummern**. `LED(17)` meint GPIO17, das ist Pin 11 der Leiste.

> **Die Pins vertragen nur 3,3 V!** Ein 5-V-Signal (zum Beispiel vom Echo-Pin eines Ultraschallsensors oder von einem Arduino) kann den Pi zerstören. Dafür braucht man einen Spannungsteiler oder einen Pegelwandler. Pro Pin sind etwa **16 mA** erlaubt, bleib bei LEDs mit 330 Ω Vorwiderstand (3,3 V − 2 V = 1,3 V, 4 mA bis 10 mA reichen für eine helle LED).

Die Pins 1 und 17 liefern 3,3 V, die Pins 2 und 4 liefern 5 V (vom Netzteil, nur als Stromversorgung, nicht als Signal). Masse (GND) gibt es an mehreren Pins, zum Beispiel Pin 6 und Pin 9.

4. Python in fünf Minuten

Auf dem Pi programmierst du meist in **Python**. Das liest sich fast wie Englisch. Statt geschweifter Klammern zählt die **Einrückung** (4 Leerzeichen).

```
name = "Azubi"                    # Variable (Text)
zahl = 7                          # ganze Zahl
print("Hallo", name, zahl)        # Ausgabe

if zahl > 5:                      # Entscheidung (Doppelpunkt!)
    print("groß")
else:
    print("klein")

for i in range(3):                # i = 0, 1, 2
    print(i)

while True:                       # Endlosschleife
    break                         # verlässt die Schleife

def verdopple(x):                 # eigene Funktion
    return x * 2
```

- **Doppelpunkt** am Ende von `if`, `for`, `while`, `def`.
- **Einrückung** zeigt, was zum Block gehört. Falsch eingerückt gibt einen Fehler.
- Warten: `from time import sleep` und `sleep(0.5)` (Sekunden, auch Kommazahlen).
- Ein Programm startest du in Thonny mit dem grünen Pfeil oder im Terminal mit `python3 programm.py`. Beenden mit `Strg+C`.

5. Erstes Projekt: LED und Taster

Die Bibliothek **gpiozero** ist auf dem Raspberry Pi OS schon installiert und macht das Ansteuern sehr einfach.

LED blinken lassen

Schaltung: GPIO17 (Pin 11) → 330-Ω-Widerstand → LED (Anode) → Kathode an GND (zum Beispiel Pin 9).

```
from gpiozero import LED
from time import sleep

led = LED(17)
while True:
    led.on()
    sleep(1)
```

```
    led.off()
    sleep(1)
```

Kurz: `led.blink()` blinkt im Hintergrund, danach mit `pause()` (aus `signal`) warten.

Taster abfragen

Schaltung: Taster zwischen GPIO2 (Pin 3) und GND. `Button` schaltet intern einen Pull-up ein, das ist wie `INPUT_PULLUP` beim Arduino.

```
from gpiozero import Button
from time import sleep

taster = Button(2)
while True:
    if taster.is_pressed:        # hier: True = gedrückt (gpiozero dreht das für dich um)
        print("gedrückt")
    sleep(0.1)
```

Taster schaltet LED (ereignisgesteuert)

```
from gpiozero import LED, Button
from signal import pause

led = LED(17)
taster = Button(2)
taster.when_pressed = led.on
taster.when_released = led.off
pause()                          # Programm läuft weiter und wartet auf Ereignisse
```

> **Unterschied zum Arduino:** `is_pressed` ist True, wenn der Taster gedrückt ist. `gpiozero` kümmert sich um den Pull-up. Beim Arduino musst du selbst LOW abfragen.

6. Mehr Bauteile

LED dimmen (PWM)

```
from gpiozero import PWMLED
from time import sleep
led = PWMLED(18)
led.value = 0.5          # 50 % Helligkeit (0.0 bis 1.0)
led.pulse()              # sanftes Pulsieren im Hintergrund
sleep(10)
```

Analoge Sensoren: der Pi hat keinen ADC

Ein Poti oder einen TMP36 kannst du **nicht direkt** an den Pi hängen. Dafür gibt es einen kleinen Wandler-Baustein (zum Beispiel **MCP3008**, SPI). Er liefert dem Pi die Messwerte als Zahl zwischen 0 und 1.

```
from gpiozero import MCP3008
poti = MCP3008(channel=0)
print(poti.value)        # 0.0 bis 1.0
```

Ultraschall-Abstand

> Der Echo-Pin des HC-SR04 liefert 5 V. Vor den Pi gehört ein **Spannungsteiler** (zum Beispiel 1 kΩ und 2 kΩ), sonst wird der Pin zerstört.

```
from gpiozero import DistanceSensor
from time import sleep
sensor = DistanceSensor(echo=24, trigger=23, max_distance=2)
while True:
    print(round(sensor.distance * 100), "cm")
    sleep(0.5)
```

Servo

```
from gpiozero import Servo
from time import sleep
servo = Servo(17)
servo.min(); sleep(1)
servo.mid(); sleep(1)
servo.max(); sleep(1)
```

Sensoren mit I²C

Viele Sensoren (Temperatur, Display, Beschleunigung) sprechen I²C über die Pins 3 (SDA) und 5 (SCL). Das lässt sich unter „Raspberry Pi Konfiguration → Schnittstellen → I2C" einschalten. Mit `i2cdetect -y 1` findest du angeschlossene Geräte.

7. Projektideen für den Pi

- **Ampel** mit drei LEDs und Fußgängertaster (`LEDBoard`, `Button`).
- **Einbruchalarm**: PIR-Sensor löst Summer aus und schreibt die Uhrzeit in eine Datei.
- **Temperaturlogger**: Sensor misst jede Minute und speichert in eine CSV-Datei, die du in Excel öffnest.
- **Webserver**: Eine kleine Webseite zeigt den Sensorwert (Bibliothek `flask`).
- **Kamera**: Fotos per Knopfdruck oder Zeitraffer.
- **Zusammen mit dem Arduino**: Der Arduino misst und sendet per USB-Kabel (Serial), der Pi zeigt die Werte an oder speichert sie.

Programm beim Start automatisch ausführen

Im Terminal `crontab -e` öffnen und am Ende eine Zeile einfügen: `@reboot python3 /home/benutzername/projekt.py &`. Beim nächsten Neustart läuft das Programm von selbst.

8. Fehler finden und sicher arbeiten

Symptom	Meist die Ursache
LED leuchtet nicht	falsche GPIO-Nummer (BCM statt Pin-Nummer!), LED falsch herum, Masse nicht verbunden
`GPIO busy` / „Pin already in use“	Ein anderes Programm (oder ein zweites Fenster) nutzt den Pin noch. Beenden mit Strg+C.
`ModuleNotFoundError`	Bibliothek fehlt: `sudo apt install python3-gpiozero`
Einrückungsfehler	Tabulator und Leerzeichen gemischt; Doppelpunkt vergessen
Pi startet nicht	Netzteil zu schwach, SD-Karte falsch beschrieben. Auf der SD-Karte mit dem Imager neu beginnen.
Sensor liefert Unsinn	5-V-Sensor an 3,3-V-Pin, fehlender Spannungsteiler oder fehlende Masse

> **Regeln für sicheres Arbeiten:**
>
> - Verdrahten nur bei **heruntergefahrenem Pi**, danach Verbindungen doppelt prüfen.
> - Kein 5-V-Signal auf einen GPIO-Pin. Nie 3,3 V und 5 V verbinden.
> - Pins nie gegeneinander oder gegen Masse kurzschließen.
> - LEDs immer mit Vorwiderstand, Motoren und Lampen nur über Transistor, Relais oder Treiberbaustein.

9. Arduino oder Pi? Und wie geht es hier weiter

Aufgabe	Besser geeignet
Sensor auslesen, LED/Motor direkt schalten	Arduino
Zeitkritisch, schnell reagieren	Arduino
Strom sparen, Batteriebetrieb	Arduino
Daten speichern, Webseite, Netzwerk, Kamera	Raspberry Pi
Viele Programme, Bildschirm, Maus	Raspberry Pi
Beides kombinieren	Arduino misst, Pi wertet aus (USB/Serial)

Hinweis zum Tonfunk Lab: Das CodeLab simuliert heute den **Arduino Uno**. Eine Simulation von Python und GPIO für den Raspberry Pi ist geplant. Bis dahin kannst du die Schaltungen (LED, Taster, Sensoren) im CodeLab mit dem Arduino testen und die Logik später in Python übertragen. Die Elektronik ist dieselbe, nur die Spannung (3,3 V statt 5 V) und die Sprache ändern sich.

Weiter geht es mit dem Lernheft Arduino verstehen.